

グラフによる 問題の表現

関係をデータにし、経路を探す

情報システム学科 3年 / 選択2単位

今日の到達目標

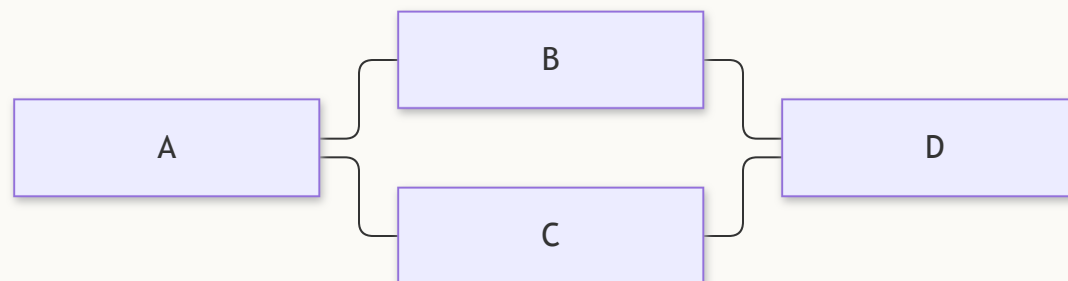
- 実際の関係を頂点と辺として表現できる
- 向きと重みの意味を、問題の条件から決められる
- BFSの訪問済み管理と経路復元を説明できる

次回の経路探索演習に使う設計を、自分の言葉で書く。

前回との接続

木の探索では、親子関係を一方向にたどった。

グラフでは、同じ頂点へ複数の経路で到達することがある。



訪問済みの管理が、重複と循環を防ぐ。

頂点と辺

グラフ $G = (V, E)$ は、頂点の集合 V と辺の集合 E で関係を表す。

題材	頂点	辺
校内の移動	場所	通れる区間
Webサイト	ページ	リンク
作業の依存	作業	前提となる関係
友人関係	人	相互の関係

何を1頂点とするかが、モデルの粒度を決める。

モデルの境界

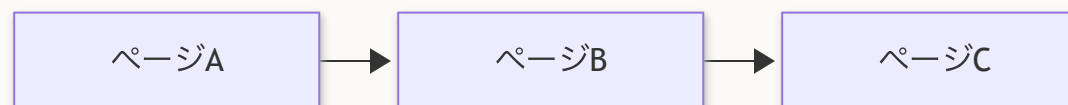
校内の「最短経路」でも、目的によって辺の意味が変わる。

- 通路の数を減らす：各辺を1区間として数える
- 移動時間を減らす：各辺に時間を持たせる
- 階段を避ける：通行可能な辺を限定する

要求を確認してから、必要なデータを決める。

有向グラフ

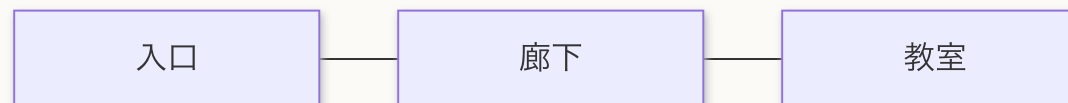
辺に向きがある。AからBへ行けても、BからAへ行けるとは限らない。



例：一方通行、ページのリンク、作業の依存。

無向グラフ

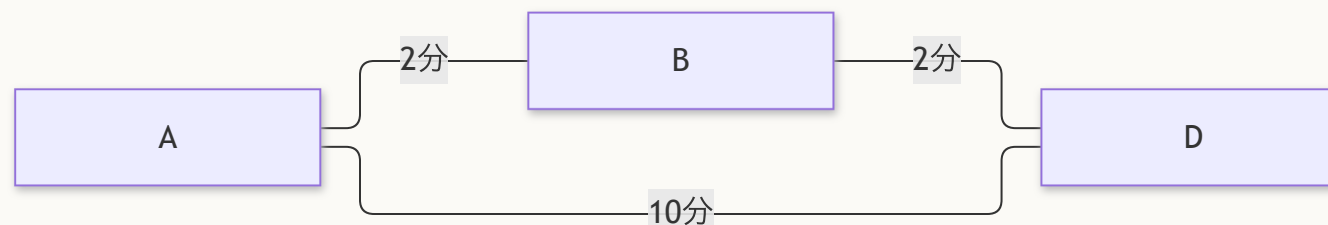
辺の両方向に移動できる。



隣接リストでは、Aの隣にBを書くと同時に、Bの隣にもAを書く。

重み付きグラフ

辺に距離・時間・料金などの値を持たせる。



AからDへは、1辺の経路が10分、2辺の経路が4分。

BFSが保証するのは、重みなし、または各辺の費用が等しい場合の最少辺数。

隣接リスト

各頂点について、直接つながる頂点を持つ。

```
graph = {  
    "A": ["B", "C"],  
    "B": ["A", "D"],  
    "C": ["A", "D"],  
    "D": ["B", "C"],  
    "X": [],  
}
```

孤立した頂点Xも明示する。頂点集合を、辺だけから推測しない。

次数

単純無向グラフでは、頂点につながる辺の数を次数と呼ぶ。

有向グラフでは、出ていく辺を **出次数**、入ってくる辺を **入次数** として数える。

```
directed = {"A": ["B"], "B": ["D"], "D": []}  
print(len(directed["A"])) # Aの出次数は1
```

次数の値に正負の符号がある、という意味ではない。

隣接行列との比較

行と列を頂点に対応させ、接続の有無を格納する。

表現	記憶量の目安	得意な操作
隣接リスト	$O(V + E)$	隣の頂点の列挙
隣接行列	$O(V^2)$	特定の2頂点の接続確認

V は頂点数、 E は辺数。無向リストでは各辺を両方向に持つ。

辺が少ないグラフでは、隣接リストを使いやすい。

経路と距離

経路は、辺で結ばれた頂点の列。

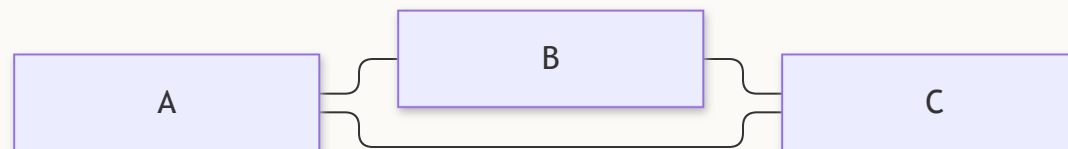
A, B, D の辺数は 2。頂点数は3。

- この回の距離は、移動した辺の数
- 最短経路が複数あることもある
- 同じ長さなら、隣接リストの順が返す経路に影響する

期待値は「長さ」と「経路の妥当性」の両方で確認する。

循環

ある頂点から辺をたどり、同じ頂点へ戻れる構造。



訪問済み进行管理しない探索は、同じ頂点を繰り返し処理することがある。

無向グラフの循環判定では、直前の親へ戻る辺と、それ以外の再訪を区別する。

連結性と到達可能性

無向グラフが連結なら、どの頂点の組にも経路がある。

- X が孤立していれば、全体は非連結
- BFSで到達した集合を求められる
- 有向グラフでは、出発点ごとに到達できる集合が変わる

有向グラフの「全頂点から互いに到達できる」は、強連結という別の条件。

探索する問題の仕様

入力：隣接リスト、出発点start、到着点goal。

出力：最少辺数の経路1本。到達できなければ `None`。

- 出発点と到着点が同じなら `[start]`
- 未知の頂点名は入力エラーとして扱う
- この演習は重みなし・無向グラフ

到達不能と入力エラーを区別する。

BFSの層

キューを使うと、出発点から近い距離の順に処理する。

距離	頂点
0	A
1	B, C
2	D
到達不能	X

未発見の頂点へ伸ばす辺は、現在の距離に1を足す。

発見時の記録

キューへ入れた時点で訪問済みにする。

```
queue = deque([start])
parent = {start: None}
while queue:
    node = queue.popleft()
    for neighbor in graph[node]:
        if neighbor not in parent:
            parent[neighbor] = node
            queue.append(neighbor)
```

`parent` は発見済み集合と、直前の頂点の記録を兼ねる。

訪問済みを記録する理由

BとCの両方からDへ行ける場合を考える。

1. Bを処理してDを発見し、キューへ入れる
2. Cを処理したとき、Dはすでに発見済み
3. Dを再びキューへ入れず、最初の親を保つ

各頂点を1回だけキューへ入れれば、循環があっても探索を終了できる。

経路の復元

到着点から親をたどり、最後に逆順にする。

```
path = []
node = goal
while node is not None:
    path.append(node)
    node = parent[node]
path.reverse()
```

D の親が B、B の親が A なら、結果は A, B, D。

未到達のgoalを先に判定してから復元する。

BFSの計算量

隣接リストで、各頂点と各隣接要素を一通り調べる。

- 時間： $O(V + E)$
- 追加領域：キューと親記録に $O(V)$
- 入力グラフ自体： $O(V + E)$

辞書・集合の操作を平均的に $O(1)$ とみなすモデル。

探索ごとに経路全体をコピーする実装では、追加コストを別に数える。

DFSを選ぶ場面

DFSは、奥の分岐を調べて戻る操作に合う。

- 到達する頂点の列挙
- 循環の検出や依存関係の検討
- 分岐を選び直す探索

DFSで最初に見つけた経路の辺数が最少とは限らない。

求めたい答えの条件から、探索法を選ぶ。

実演の実行

Python 3.12以上の環境で実行する。

```
python3 examples/05-graphs/graphs.py
```

出力でBFSとDFSの訪問順、経路、連結成分を確認する。

経路が出たら、各隣接頂点の組が元の辺に含まれるかを図と照合する。

辺を1本削った場合の結果を、変更前に予測する。

演習1：モデルの作成

`exercises/05.md` の架空校内マップを使う。

- 5頂点以上、孤立頂点を1つ含む隣接リストを作る
- 無向辺を両方向に登録する
- 自分のモデルで「1辺」の意味を書く

成果物： `05-map.py` と、頂点・辺・前提を記した `05-design.md`。

演習2：探索の追跡

AからDを探すBFSを、1回の取り出しごとに記録する。

記録する内容：取り出した頂点、残るキュー、新しく記録した親。

期待する性質：最初の発見で親が決まり、同じ頂点が重複して入らない。

AIに説明を依頼した場合も、キューの変化は自分で図と照合する。

確認問題

1. 一方通行を無向辺として登録すると、どんな誤答が起こるか
2. BFSで訪問済みを記録するタイミングはどこか
3. 最少辺数と最短時間が一致する条件は何か
4. 孤立頂点を持つ入力で、到達不能をどう返すか

解答では、前提条件と理由を添える。

まとめと次回

グラフの設計では、頂点・辺・向き・重みの意味を先に決める。

BFSは、等しい費用の辺で最少辺数の経路を探せる。親の記録を使うと、その経路を復元できる。

次回は、仕様を満たす経路探索プログラムを実装し、授業内で提出する。

参考資料

- [Python公式：collections.deque](#)
- [MIT 6.006：BFS・DFSの講義資料](#)

図・校内マップ・コードは本教材用の自作。

次回の課題手順： `exercises/06.md`。